

Claude Code 세션 관리

Session · Fork · Checkpoint 정리

작성일: 2026년 6월 21일

1. Session(세션)이란

Claude Code에서 "세션"은 하나의 터미널 인스턴스에서 시작해서 종료될 때까지의 대화 흐름 전체를 의미한다. `claude` 명령어를 실행하면 새 세션이 시작되며, 세션 안에서 주고받는 모든 메시지(사용자 프롬프트, Claude 응답, 도구 호출 기록)가 하나의 컨텍스트로 누적된다. 세션은 `~/.claude/projects/<project-id>/` 아래에 JSONL 파일로 저장되며, 파일명이 곧 세션 ID가 된다.

1.1 세션의 경계를 결정하는 동작

동작	세션에 미치는 영향
<code>/clear</code>	현재 세션의 컨텍스트를 비움 (같은 프로세스, 새 세션처럼 시작)
<code>/compact</code>	세션은 유지하되 컨텍스트를 요약·압축
터미널 종료	세션 종료, 단 히스토리는 디스크에 남음
<code>claude --continue</code>	가장 최근 세션을 바로 이어감
<code>claude --resume <id></code>	특정 세션 ID로 정확히 복귀

1.2 주요 명령어

- `claude --resume` : 세션 목록을 대화형으로 표시 (날짜·첫 메시지 미리보기 포함)
- `claude --resume <session-id>` : 특정 세션으로 직접 재개
- `claude --continue` : 가장 최근 세션을 선택 없이 바로 이어감
- `/clear` : 컨텍스트 초기화 (새 작업 시작 시)
- `/compact` : 컨텍스트 압축 요약 (긴 작업 중 토큰 절약)
- `/resume` : 세션 안에서 다른 세션으로 전환
- `/cost` : 현재 세션의 토큰/비용 확인

1.3 GD RDM 프로젝트 적용 팁

Feature 단계별로 작업 단위를 세션으로 나누는 것이 좋다. 새 단계(예: Feature 3 분석 로직) 시작 시 `/clear`로 컨텍스트를 비우면 토큰을 절약하고 혼선을 줄일 수 있다. 단, CLAUDE.md에 진행 상황을 잘 기록해두어야 다음 세션이 맥락을 빠르게 따라잡을 수 있다.

2. `/fork` — 세션 분기

지금까지의 대화 히스토리(컨텍스트) 전체를 복사하여, 원본은 그대로 둔 채 새 세션 ID로 독립적인 분기를 만드는 명령어다. 핵심은 컨텍스트는 분기되지만 파일(작업 디렉터리)은 공유된다는 점이다. 포크된 세션이 파일을 수정하면 그 변경은 실제로 적용되며, 같은 디렉터리에서 작업하는 다른 세션에도 그대로 보인다.

2.1 사용법

- 세션 안에서 바로: `/fork`
- CLI 에서 재개하면서: `claude --resume <session-id> --fork-session`
- 가장 최근 세션을 포크: `claude --continue --fork-session`

2.2 활용 시나리오

시나리오	설명
위험한 리팩토링 전 체크포인트	리스크 있는 변경을 시도하기 전 안전하게 실험할 분기를 만들어 둔다.
A/B 프롬프트 구현 비교	동일한 마스터 세션을 두 번 포크해 서로 다른 접근을 시도하면, 결과 차이가 컨텍스트 드리프트가 아닌 순수한 접근 방식 차이에서 비롯됨을 보장할 수 있다.
디버깅 중 대안 경로 탐색	제안된 수정안과 다른 경로를 시도하고 싶을 때, 기존 작업을 잃을 위험 없이 분기해서 시도하고 실패하면 원본 세션으로 복귀한다.

2.3 주의사항

파일은 세션 간에 공유된다. 두 분기를 동시에 같은 디렉터리에서 실행하면 파일 수정이 서로 충돌할 수 있다. 완전히 격리된 실험이 필요하다면 `git worktree`로 별도 디렉터리까지 분리하는 것이 안전하다.

3. Checkpoint — 파일 스냅샷과 `/rewind`

체크포인트는 Claude가 파일을 수정하기 직전마다 자동으로 만드는 스냅샷이다. Git

커밋과 달리 의도적으로 찍는 것이 아니라, 매 편집마다 자동으로 기록되는 세션 단위 안전망이다. 기본적으로 활성화되어 있어 별도 설정이 필요 없다.

3.1 핵심 제약: 무엇이 기록되고 무엇이 안 되는가

Write, Edit, NotebookEdit 도구를 통한 파일 수정만 추적된다. Bash 명령어(echo >, sed -i, 스크립트 직접 실행 등)를 통한 변경은 체크포인트 시스템에 잡히지 않는다. 따라서 GD RDM 프로젝트처럼 python rdm_merge.py 같은 스크립트를 bash로 직접 실행해 결과 파일을 생성·덮어쓰는 경우, 그 변경은 체크포인트로 복구되지 않는다. 중요한 단계 전에는 git commit을 별도로 남겨두는 것이 안전하다.

3.2 불러오는 방법

- Esc 키 두 번 연속 입력 (Esc Esc)
- 또는 /rewind 명령어 입력

3.3 복원 옵션

옵션	효과	언제 사용하나
Code only (코드만)	파일만 되돌리고 대화는 유지	실행은 실패했지만 Claude의 맥락 이해는 맞을 때
Conversation only (대화만)	코드 변경은 유지한 채 대화 컨텍스트만 리셋	코드는 맞는데 이후 대화가 산으로 갈 때
Both (둘 다)	파일과 메시지를 모두 해당 체크포인트로 복원	가장 흔히 쓰는, 가장 안전한 선택

추가로 "Summarize from here" 옵션도 있다. 해당 시점 이후의 대화를 AI가 요약·압축하는 기능으로, 파일은 건드리지 않고 컨텍스트 윈도우 공간만 절약한다.

3.4 활용 패턴

- 여러 접근법 비교: 체크포인트로 되돌아가 다른 방법을 시도하고, 결과를 비교한 뒤 가장 좋은 것을 선택
- 실수 복구: 원치 않는 변경을 되돌리거나, 에이전트가 잘못된 수정을 했을 때 안전한 상태로 복원

3.5 주의사항 — 인지 불일치

코드만 되돌리면 Claude의 인식과 실제 파일 상태 사이에 불일치가 생긴다. 다음 지시를 줄 때 "이전 변경을 롤백했다"는 점을 명시적으로 알려줘야 한다. 특별한

이유가 없다면 코드와 대화를 모두 복원(Both)하는 것이 안전하다.

4. /fork vs Checkpoint(/rewind) 비교

구분	/fork	Checkpoint (/rewind)
대상	대화 컨텍스트 분기	파일 + (선택적으로) 대화 상태 복원
결과	원본·분기 세션이 모두 살아있음 (병렬 구조)	과거 시점으로 되돌림 (선형 구조)
용도	A/B 두 접근을 동시에 살려두고 비교	잘못된 방향으로 갔을 때 되돌리기
파일시스템	세션 간 공유 (분기해도 파일은 하나)	복원 시 디스크의 파일이 실제로 되돌아감

5. GD RDM 프로젝트 적용 요약

- Feature 단계별 작업은 세션 단위로 나누고, 새 단계 시작 시 /clear 로 컨텍스트 정리
- 두 가지 구현 방식을 비교하고 싶을 때(예: pandas groupby vs numpy 벡터화)는 /fork 로 분기하여 병렬 비교
- Claude 의 파일 편집(Write/Edit)이 잘못됐을 때는 Esc Esc 또는 /rewind 로 즉시 복구
- bash 로 직접 실행하는 스크립트(rdm_merge.py 등)는 체크포인트 보호를 받지 못하므로, 중요한 단계 전 git commit 을 습관화
- /rewind 는 git 을 대체하지 않는 보조 안전망이라는 점을 유념